ChatGPT

HiDiffusion 消融实验矩阵执行计划

执行摘要

本计划把你的现有 benchmark runner 扩展为**消融实验矩阵**：以 `baseline / HiDiffusion × (xformers, cpu_offload, vae_tiling)` 为因子，系统评测不同优化组合在 **latency / throughput / peak VRAM / failure rate** 上的叠加效应与 trade-off，并生成可写简历的工程评测产物（`ablation.csv`、`ablation_report.md`、图表与样例图）。
实现不涉及训练新模型，全部基于 diffusers 推理开关与 HiDiffusion 的“一行集成”（`apply_hidiffusion(pipe)`）完成。HiDiffusion README 也明确提到 xFormers、model CPU offload、VAE tiling 作为可选节省显存/加速手段。[1]
关键工程要点是：**CPU offload 是 stateful hooks 操作**，需要避免在同一进程反复切换造成残留；因此本计划采用“**每个实验配置单独进程/单独 pipeline 实例**”的运行策略，并把失败模式纳入统计。[2]

目标产出与验收标准

可量化期望产出

必交付（写进简历/README 的核心）：

- `results/ablation/ablation.csv`：全量明细数据（每次运行一行），含 latency/throughput/peak_vram/失败原因等
- `results/ablation/ablation_report.md`：聚合报告（均值、95%CI、成功率、主效应/交互效应解读、推荐配置）
- `results/ablation/figures/`：至少 4 张图
- latency（按分辨率分组的组合柱状图）
- peak VRAM（按分辨率分组的组合柱状图）
- success rate（分辨率-成功率折线图）
- Pareto/Trade-off（latency vs VRAM 的散点图，按组合着色）
- `results/ablation/samples/`：每个关键组合至少 1 张示例图片（固定 prompt/seed）
- `experiments/ablation_manifest.yaml`：实验清单（矩阵定义、优先级、重复次数、参数）
- `scripts/ablation/`：执行脚本（manifest 生成、单实验运行、跑批 orchestrator、汇总与画图）
- `README.md`：一键复现实验与复现报告命令（本地/Colab/云 GPU）

“完成且可写简历”的验收标准

你可以用一句话证明项目完成并可复现：

1) 新机器按 README 命令能跑出 `ablation.csv` 与 `ablation_report.md`；
2) 报告中至少给出：
- **baseline vs HiDiffusion 的收益**（在某分辨率点 latency/VRAM 提升百分比）
- 至少一项**组合效应结论**（例如“xformers 与 HiDiffusion 可叠加/存在边际收益递减，且在高分辨率更明显”）
- **失败模式**（OOM/xformers 不可用/offload hooks 相关/加载失败）与成功率曲线
3) 你能回答面试问题：

- “收益来自 HiDiffusion 还是来自 xformers/offload/tiling？”
- “为什么 offload 必须单进程/单 pipeline 隔离？”

实验矩阵清单

因子定义

- 因子 1：`hidiffusion ∈ {0,1}`（`apply_hidiffusion(pipe)` 一行集成） [1]
- 因子 2：`xformers ∈ {0,1}`（`pipe.enable_xformers_memory_efficient_attention()`；diffusers 推荐 xFormers 可提升速度并降显存） [3]
- 因子 3：`cpu_offload ∈ {0,1}`（`pipe.enable_model_cpu_offload()`；官方明确是 **stateful hooks**，需按 pipeline 预期顺序执行并谨慎复用） [2]
- 因子 4：`vae_tiling ∈ {0,1}`（`pipe.enable_vae_tiling()`；官方说明可显著省显存并支持更大图像，但可能有 tile-to-tile tone variation；<=512×512 会自动禁用） [4]

  注意：HiDiffusion README 自己也把 xformers / model_cpu_offload / vae_tiling 作为“可选优化开关”写在示例注释里，这正是你要做消融矩阵的直接依据。 [5]

推荐“优先级分层”的矩阵（按 P0→P2）

为兼顾算力/时间，本计划把实验矩阵按价值与必要性排序。

P0：最小但信息量最大（建议先跑完，形成主结论）

共 8 组（与你提出的一致），每组在 3 分辨率 × 2 seeds × 3 repeats 下运行：

1. baseline
2. baseline + xformers
3. baseline + cpu_offload
4. baseline + vae_tiling
5. HiDiffusion
6. HiDiffusion + xformers
7. HiDiffusion + xformers + vae_tiling
8. HiDiffusion + xformers + cpu_offload

用途：
- 直接回答“收益归因”与“组合叠加效应”。
- 覆盖最常见的工程开关组合（尤其 xformers + tiling 常被建议搭配使用）。 [6]

P1：补齐 2 个关键交互（当你发现结论不稳定或某项影响很大时加跑）

1. HiDiffusion + vae_tiling（不带 xformers）
2. HiDiffusion + cpu_offload（不带 xformers）

用途：拆分 “HiDiffusion+tiling” 与 “HiDiffusion+offload” 的纯贡献，避免把收益都归因到 xformers。 [7]

P2：可选扩展（用于写“更专业”的 trade-off 章节）

1. baseline + xformers + vae_tiling
2. baseline + xformers + cpu_offload

3. baseline + vae_tiling + cpu_offload
4. HiDiffusion + vae_tiling + cpu_offload
5. HiDiffusion + xformers + vae_tiling + cpu_offload（全开）

用途：做“全组合 Pareto 前沿”更完整，但耗时显著增加（建议在有云 GPU 或本地算力充裕时做）。

实验清单与 manifest 生成

建议的文件与目录结构

```
experiments/
  ablation_manifest.yaml
  promptset_v1.json

scripts/
  ablation/
    gen_manifest.py
    run_one.py
    run_manifest.py
    aggregate.py
    plot.py
    utils_env.py

results/
  ablation/
    runs/
      20260322_153000/          # 每次跑批一个 run_id 目录
        raw.csv
        env.json
        logs/
        samples/
    ablation.csv                 # 合并后的全量明细
    ablation_report.md
    figures/
    recommended_strategy.json     # 可选：从结果导出建议配置
```

manifest 文件格式建议

文件名：`experiments/ablation_manifest.yaml`

字段建议（确保可复现与可追踪）：

- `run_id`（可由脚本生成）
- `model_id`（如 `stabilityai/stable-diffusion-xl-base-1.0`；HiDiffusion 支持 SDXL/SDXL-Turbo/SD2/SD1，README 列明支持模型）[8]
- `pipeline_type`（如 `DiffusionPipeline` / `AutoPipelineForText2Image`；你可固定一种以减少变量）
- `scheduler`（如 `ddim`，或额外加 `euler_a` 做扩展）
- `promptset_path`（JSON）
- `matrix`：每个实验条目包含：

- `exp_id`
- `priority`（P0/P1/P2）
- `hidiffusion`（0/1）
- `xformers`（0/1）
- `cpu_offload`（0/1）
- `vae_tiling`（0/1）
- `height,width`（列表或单值）
- `steps`（固定或列表）
- `seeds`（列表）
- `repeats`（>=3）

manifest 生成脚本示例（关键代码片段）

`gen_manifest.py`（简化示意）：

```
import yaml, itertools, datetime

P0 = [
  ("baseline",                    0,0,0,0),
  ("base+xformers",               0,1,0,0),
  ("base+offload",                0,0,1,0),
  ("base+tiling",                 0,0,0,1),
  ("hidiffusion",                 1,0,0,0),
  ("hi+xformers",                 1,1,0,0),
  ("hi+xformers+tiling",          1,1,0,1),
  ("hi+xformers+offload",         1,1,1,0),
]

def build_manifest():
    run_id = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    manifest = {
      "run_id": run_id,
      "model_id": "stabilityai/stable-diffusion-xl-base-1.0",
      "scheduler": "ddim",
      "promptset_path": "experiments/promptset_v1.json",
      "resolutions": [{"h": 1024, "w": 1024}, {"h": 1536, "w": 1536}, {"h": 2048, "w": 2048}],
      "steps": 20,
      "seeds": [0,1],
      "repeats": 3,
      "matrix": []
    }
    for name, hi, xf, off, til in P0:
        manifest["matrix"].append({
          "exp_id": name,
          "priority": "P0",
          "hidiffusion": hi,
          "xformers": xf,
          "cpu_offload": off,
          "vae_tiling": til
        })
    return manifest
```

```
if __name__ == "__main__":
    m = build_manifest()
    with open("experiments/ablation_manifest.yaml", "w") as f:
        yaml.safe_dump(m, f, sort_keys=False, allow_unicode=True)
```

每个实验的运行编排与命令模板

为什么建议“每组配置一个进程”

- `enable_model_cpu_offload()` 在 diffusers 中被官方明确为 **stateful operation**：会在模型上安装 hooks，并要求按 pipeline 的预期顺序调用；若你复用模型组件或者在同一进程里频繁切换，很容易留下 hook 状态，导致不可预测结果。 [2]
- 因此，最稳的工程做法是：**每个配置使用独立 pipeline 实例，最好独立进程**。这也更像真实“评测基建”的运行方式（可重放、可隔离、失败不污染全局）。

单次实验运行命令模板（run_one.py）

建议 `scripts/ablation/run_one.py` 支持参数：

- `--model_id`
- `--scheduler` （如 `ddim`）
- `--hidiffusion 0/1`
- `--xformers 0/1`
- `--cpu_offload 0/1`
- `--vae_tiling 0/1`
- `--height --width --steps`
- `--seed`
- `--repeat_idx`
- `--out_dir results/ablation/runs/<run_id>/`
- `--prompt_id` （从 promptset 里取固定某条，避免 prompt 变成扰动源）

示例：

```
python scripts/ablation/run_one.py \
  --model_id "stabilityai/stable-diffusion-xl-base-1.0" \
  --scheduler "ddim" \
  --hidiffusion 1 \
  --xformers 1 \
  --cpu_offload 0 \
  --vae_tiling 1 \
  --height 1536 --width 1536 \
  --steps 20 \
  --seed 0 \
  --repeat_idx 1 \
  --prompt_id 0 \
  --out_dir "results/ablation/runs/20260322_153000"
```

跑批命令（run_manifest.py）

`run_manifest.py` 读取 `ablation_manifest.yaml`，对每个矩阵项展开成：

`(exp_id × resolution × seed × repeat × prompt_id)` 的任务队列，并用 `subprocess` 调度 `run_one.py`。

简化示意：

```
import subprocess, yaml

def run_job(job):
    cmd = ["python", "scripts/ablation/run_one.py", *job.to_cli_args()]
    subprocess.run(cmd, check=False)  # 不因失败中断；失败在CSV里记录

# 主流程：读manifest → 展开jobs → 顺序或并行执行
```

关于 xformers / offload / tiling 的“安全切换”规则

- **xformers**：需要安装 xFormers 后启用；diffusers 文档推荐 xFormers 用于低内存更快推理，并提醒 pip 包通常要求较新的 PyTorch。建议启用失败时写入 `failure_reason=xformers_unavailable` 并继续实验。 [3]
- **cpu_offload**：必须独立 pipeline；不要在一个 pipeline 上“启用后又禁用再启用”来切换组合。官方强调其 hooks 与调用顺序依赖。 [2]
- **vae_tiling**：同理建议每个组合新建 pipeline；并在报告说明可能带来的输出 tone variation（这是官方已提醒的副作用）。 [9]

数据采集字段（CSV schema）与样例行

ablation.csv 推荐 schema

最少必须包含（满足你原 benchmark 的工程指标与消融分析需求）：

- 识别信息
- `run_id, exp_id, priority, model_id, scheduler, prompt_id, seed, repeat_idx`
- 配置因子
- `hidiffusion, xformers, cpu_offload, vae_tiling`
- 参数
- `height, width, steps, guidance_scale, dtype`
- 指标（核心）
- `latency_sec`（单张耗时）
- `throughput_img_s`（=1/latency，或在多张批量时定义更合理）
- `peak_vram_mb`（`torch.cuda.max_memory_allocated()`）
- `success`（0/1）
- `failure_reason`（枚举）
- 环境（可复现关键）
- `torch_version, diffusers_version, hidiffusion_version, xformers_version, cuda_version, gpu_name`

样例行（示意）

```
run_id,exp_id,priority,model_id,scheduler,prompt_id,seed,repeat_idx,hidiffusion,xformers,cpu_offload,vae_tiling,h
20260322_153000,hi+xformers+tiling,P0,stabilityai/stable-diffusion-xl-base-1.0,ddim,
0,0,1,1,1,0,1,1536,1536,20,7.82,0.128,10934,1,,NVIDIA RTX 4090,0.3x.x
```

最少实验规模建议与算力折中

你要求的最少规模（用于“结论可信 + 可写简历”）

- 分辨率：至少 3 个点
- seeds：至少 2 个
- repeats：每个 (exp_id, resolution, seed, prompt) 至少 3 次（避免偶然波动）
- prompt：建议固定 1 条主 prompt（减少变量），另加 1 条备选 prompt 用于 sanity check

推荐默认（根据模型与算力分档）

- 若用 SDXL（HiDiffusion 明确支持 SDXL）：推荐分辨率 `1024，1536，2048`（更能体现高分辨率/显存压力差异）。 [10]
- 若算力不足：用 SD1.5 或 SD2.1（HiDiffusion 也支持）：分辨率 `512，768，1024`，把“失败率曲线”作为主要亮点。 [8]

  VAE tiling 在 512×512 或更小会自动禁用（官方说明），因此如果你要观察 tiling 的收益，分辨率至少要 >512。 [9]

结果分析方法

统计汇总与置信区间

对每个 `(exp_id, resolution)` 计算：

- `mean_latency`、`std_latency`
- 95% CI（小样本用 t 分布）：
- `CI = mean ± t_{0.975, n-1} * (std / sqrt(n))`

同理对 peak_vram 与 success_rate 做汇总（success_rate 用二项分布近似也行；two-proportion CI 可选）。

归因分析（主效应 + 交互效应）

推荐两层分析（两周内都可做）：

1) **差分归因（易解释）**
- HiDiffusion 主效应：`(Hi - baseline)` 在相同 (xformers/offload/tiling) 条件下的差
- xformers 主效应：`(xformers on - off)` 在固定 HiDiffusion 开关下的差
- 交互：
- `Δ_hi_when_xf_on - Δ_hi_when_xf_off`（判断叠加是否存在互补/替代）

2) **线性模型/ANOVA（更“专业”，但可选）**
- 令 `y = log(latency)`，用因子模型：
- `y ~ hi + xf + off + til + hi:xf + hi:off + hi:til + ...`

- 解释：哪些因子/交互项显著、方向如何。
- 这能让你在报告里写出“统计上更严谨”的结论（若你时间够）。

绘图展示组合效应与 trade-off

最低建议 4 张图：

- 分辨率分组条形图（latency）
- 分辨率分组条形图（peak_vram）
- 分辨率 vs success_rate（折线图）
- Pareto 图：x 轴 peak_vram，y 轴 latency（越左下越好），标注 P0 组合

绘图伪代码（plot.py 思路）：

```
# 伪代码：读 ablation.csv → groupby 聚合均值/CI → matplotlib 画图
df = read_csv("results/ablation/ablation.csv")
agg = df[df.success==1].groupby(["exp_id","height","width"]).agg(
    mean_latency=("latency_sec","mean"),
    ci_latency=("latency_sec", lambda x: t_ci(x)),
    mean_vram=("peak_vram_mb","mean"),
    success_rate=("success","mean"),
)
barplot_by_resolution(agg, metric="mean_latency")
scatter_pareto(agg, x="mean_vram", y="mean_latency", hue="exp_id")
```

质量与稳定性检查（failure_reason 与 success rate）

failure_reason 分类（建议固定枚举）

- `oom`：`torch.cuda.OutOfMemoryError`
- `xformers_unavailable`：未安装/版本冲突/启用失败（xFormers 安装与版本依赖见官方文档提示）[3]
- `cpu_offload_hook_error`：offload hooks 相关异常（stateful + 调用顺序依赖是官方明确警告点）[2]
- `model_load_failure`：模型下载/鉴权/文件缺失
- `invalid_config`：分辨率/steps 参数非法
- `other_exception:<Type>`：兜底

success rate 分析

- 按 `(exp_id, resolution)` 计算成功率
- 画出“分辨率-成功率曲线”，并在报告写一个章节：
- **Where it helps**：哪类场景提升最大
- **Where it fails**：失败集中在什么组合/分辨率（例如 offload 更稳但更慢，或某组合更容易 OOM）

依赖与硬件假设（含 Colab/云替代）

最低可行（用于跑通流程与小规模对比）

- GPU：4–8GB（只能跑低分辨率/低 steps；重点做流程+失败模式）

- CPU：足够 RAM（>=16GB 推荐）
- Python：3.9–3.11
- 关键包：`torch`，`diffusers`，`transformers`，`accelerate`，`hidiffusion`，`safetensors`，`pandas`，`pyyaml`，`matplotlib`

  CPU offload / model offloading 需要 accelerate 支持，diffusers 文档在“Reduce memory usage/Model offloading”章节中明确依赖与用法。[11]

推荐硬件（用于形成漂亮结论）

- GPU：≥12GB（更稳跑 SDXL 1024–1536；2048 视组合而定）
- 若要完整跑 P0 的 2048×2048 + repeats：16–24GB 更舒适

Colab/云 GPU 快速跑批示例（命令示意）

```
pip install -U diffusers transformers accelerate safetensors hidiffusion pandas pyyaml matplotlib
# xformers 可选，能装则加：pip install xformers

python scripts/ablation/gen_manifest.py
python scripts/ablation/run_manifest.py --manifest experiments/ablation_manifest.yaml
python scripts/ablation/aggregate.py --run_dir results/ablation/runs/<run_id>
python scripts/ablation/plot.py --csv results/ablation/ablation.csv
```

交付物清单与 README 一键命令

交付物清单（路径级别）

- `experiments/ablation_manifest.yaml`
- `experiments/promptset_v1.json`
- `scripts/ablation/gen_manifest.py`
- `scripts/ablation/run_one.py`
- `scripts/ablation/run_manifest.py`
- `scripts/ablation/aggregate.py`
- `scripts/ablation/plot.py`
- `results/ablation/ablation.csv`
- `results/ablation/ablation_report.md`
- `results/ablation/figures/*.png`
- `results/ablation/samples/*.png`
- `results/ablation/runs/<run_id>/env.json`
- `README.md`：包含“一键跑 P0”“低算力 smoke test”“xformers/offload/tiling 说明与注意事项”

README 一键命令（建议提供三档）

- `make ablation_p0_smoke`：低分辨率 + 1 seed + 1 repeat（验证链路）
- `make ablation_p0`：3 分辨率 + 2 seeds + 3 repeats（正式）
- `make report`：从 raw.csv 生成 ablation.csv + report + figures

预估时间表（80 小时上限）与兼职估时

以你已具备基础 benchmark runner 为前提（否则需额外 +8–12h）：

- 实验矩阵与 manifest：6–10h
- run_one 安全开关与失败分类：10–16h
- run_manifest 跑批 + 日志/目录结构：8–12h
- 跑 P0 全量实验（取决于 GPU）：
- 本地 12–24GB：约 8–20h（主要是机器跑，不是人力）
- 低算力：只能缩小规模或上云
- 汇总分析与报告（CI/主效应/交互解读）：10–16h
- 绘图与 README/简历材料：8–12h

总人力：约 42–68h（在 80h 上限内）。
兼职估时（每天 2–3h）：约 3–4 周更稳。

风险与缓解（至少六项）

1) **模型权重下载/鉴权失败**（HF 权限、网络）
- 缓解：提前验证 `model_id` 可访问；准备备选模型（HiDiffusion 支持 SD1/SD2/SDXL）。 [8]
2) **OOM 导致矩阵跑不全**
- 缓解：把 P0 先改成更低分辨率；使用 vae_tiling/offload 作为“可跑性手段”；失败也要入表并分析成功率。 [7]
3) **xformers 安装失败/版本不兼容**
- 缓解：xformers 设为可选；启用失败归类为 `xformers_unavailable`；报告写明环境限制。 [3]
4) **cpu_offload hooks 残留导致结果不可信**
- 缓解：每个配置独立进程/独立 pipeline；不在同一对象上反复 enable/disable。官方已明确 offload 是 stateful hooks。 [2]
5) **计时不准确（CUDA 异步、warmup）**
- 缓解：加入 `torch.cuda.synchronize()`；固定 warmup 次数不计入；每组 ⩾3 repeats。
6) **vae_tiling 影响输出观感引发质疑**
- 缓解：报告引用官方说明 “可能有 tile tone variation，但不应有明显 seam”；在样例图里放大展示并说明 trade-off。 [9]
7) **实验规模过大拖垮两周计划**
- 缓解：严格按 P0→P1→P2；P0 足以写简历，P1/P2 作为可选增量。

Mermaid 流程图（实验执行与产物链路）

```
flowchart TD
  A[定义实验矩阵 P0/P1/P2] --> B[生成 ablation_manifest.yaml]
  B --> C[run_manifest 展开任务队列]
  C --> D[每个配置独立进程 run_one]
  D --> E[采集 raw.csv + env.json + samples]
  E --> F[aggregate 聚合 ablation.csv]
  F --> G[统计: 均值/95%CI/成功率/失败分布]
  G --> H[plot 生成 figures]
  H --> I[输出 ablation_report.md + README 一键复现]
```

关键代码片段（计时/显存、CSV 写入）

单次运行计时与峰值显存（核心逻辑）

（示意代码；你可直接放进 `run_one.py`）

```
import time, math, torch

def measure(pipe, call_kwargs):
    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()

    t0 = time.perf_counter()
    try:
        out = pipe(**call_kwargs)
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        t1 = time.perf_counter()

        latency = t1 - t0
        peak_mb = (torch.cuda.max_memory_allocated() / (1024**2)) if
torch.cuda.is_available() else math.nan

        return True, "", latency, (1.0/latency if latency>0 else math.nan), peak_mb
    except torch.cuda.OutOfMemoryError:
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
        return False, "oom", math.nan, math.nan, math.nan
    except Exception as e:
        return False, f"other_exception:{type(e).__name__}", math.nan, math.nan, math.nan
```

CSV 写入示例

```
import csv

FIELDS = [
  "run_id","exp_id","priority","model_id","scheduler","prompt_id","seed","repeat_idx",
  "hidiffusion","xformers","cpu_offload","vae_tiling",
  "height","width","steps",
  "latency_sec","throughput_img_s","peak_vram_mb",
  "success","failure_reason",
  "torch_version","diffusers_version","hidiffusion_version","xformers_version","gpu_name"
]

def append_row(csv_path, row_dict):
    write_header = not os.path.exists(csv_path)
    with open(csv_path, "a", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS)
```

```
        if write_header:
            w.writeheader()
        w.writerow(row_dict)
```

简历条目模板与面试要点

简历条目模板（把消融矩阵写出“工程味”）

- 构建 HiDiffusion 推理优化**消融实验矩阵**（baseline/HiDiffusion × xformers/cpu_offload/VAE tiling），在 3 分辨率 × 2 seeds × 3 repeats 下评测 **latency / throughput / peak VRAM / success rate**，自动生成 `ablation.csv` 与 `ablation_report.md` 及可视化图表；分析优化手段的**主效应与交互效应**，总结不同分辨率下的最优配置与 trade-off。
- 工程化实践：采用“每配置独立进程/独立 pipeline”规避 **stateful offload hooks** 造成的实验污染，并建立 failure taxonomy（OOM/xformers 不可用/offload hooks/加载失败），量化推理稳定性与失败模式（更贴近真实推理服务评估）。 [2]

三个高频面试问题（答题要点）

1) **你怎么证明收益来自 HiDiffusion 而不是 xformers/tiling/offload?**
- 用消融矩阵分离主效应与交互效应；在相同开关条件下只切 `hidiffusion`，并用差分量化贡献。

2) **为什么不能在同一进程里反复开关 cpu_offload?**
- diffusers 官方把 `enable_model_cpu_offload()` 描述为 stateful hooks 操作，依赖 pipeline 调用顺序并可能污染复用组件；因此用独立进程/独立 pipeline 隔离。 [2]

3) **vae_tiling 节省显存但可能带副作用，你怎么处理?**
- 引用官方说明：tiling 可显著省显存并支持更大图像，但可能出现 tile tone variation；报告里把质量风险作为 trade-off 说明，并在样例图中展示。 [9]

以上计划按“可执行、可量化、可复现、可写简历”的工程标准拆解了消融实验矩阵实现路径，并用 HiDiffusion README 与 diffusers 优化文档作为依据（参考：`https://github.com/megvii-research/HiDiffusion`；`https://huggingface.co/docs/diffusers/v0.25.1/optimization/memory`；`https://huggingface.co/docs/diffusers/v0.27.1/en/optimization/memory`；`https://github.com/huggingface/diffusers/blob/main/docs/source/en/optimization/xformers.md`）。 [12]

---

[1] [5] [8] [10] [12] GitHub - megvii-research/HiDiffusion: [ECCV 2024] HiDiffusion: Increases the resolution and speed of your diffusion model by only adding a single line of code! · GitHub
https://github.com/megvii-research/HiDiffusion

[2] Reduce memory usage · Hugging Face
https://huggingface.co/docs/diffusers/v0.27.1/en/optimization/memory

[3] diffusers/docs/source/en/optimization/xformers.md at main · huggingface/diffusers · GitHub
https://github.com/huggingface/diffusers/blob/main/docs/source/en/optimization/xformers.md

[4] [6] [7] [9] Reduce memory usage · Hugging Face
https://huggingface.co/docs/diffusers/v0.25.1/optimization/memory

[11] Reduce memory usage · Hugging Face
https://huggingface.co/docs/diffusers/en/optimization/memory